

FPGA Lab – Memory Hierarchy

In this laboratory exercise, we will expand MicroBlaze's IO subsystem with a timer, create a memory hierarchy using 512MB of DDR3, and enable MicroBlaze's advanced processing elements such as floating point units. The upgraded hardware platform will then be used study the impact of the memory hierarchy on a matrix-matrix multiplication problem.

Objectives

By the end of this lab you should be able to

1. Expand the IO bus with a timer for measuring application execution time.
2. Add a DDR3 memory controller and interface to MicroBlaze via a data cache.
3. Study the performance of a matrix-matrix multiplication example.

Implementation guidelines

Use the previous Vivado project as a starting point.

First, add an AXI timer to the IO bus. To add multiple peripherals to the IO bus, multiplexed access is needed. AXI infrastructure IP, such as crossbars and converters provide the necessary functionality to seamlessly attach peripherals. You can use block automation or manually configure the interconnect block that connect the timer to MicroBlaze.

Next we'll add a DDR3 memory controller and Data Cache Interface. The Zybo Z7 board comes with 1GB of DDR3 memory (512MB usable for this lab), but it is connected to the hardened memory controller in the ZYNQ processing system. Copy the `add_ext_memory.tcl` script to the `scripts/` folder and source it using `./scripts/add_ext_memory.tcl` (please note the relative path) to update the block diagram with the pre-configured memory controller. The block diagram should convert the zynq module into the hierarchal block design element shown in Figure 1.



Figure 1: The ZYNQ block is transformed into a hierarchical block diagram

Validate the design, generate the bitstream and export the .xsa file.

Software

Once again update the hardware platform with the new .xsa, and create a new project. Copy the provided C source files for the timer (timer.c/timer.h) and the naive matrix multiplication (matmul.c/matul.h). Run some tests using the following options:

1. Optimise the code O0 → O3
2. Enable / Disable the caches to see their impact

Hint: Before running your benchmarks consider invalidating the cache. Small data arrays may already reside in the cache at the start of the experiment. This can happen if they were randomly generated in the previous step.